

CreditNirvana PS1: Real-Time Fake PTP Detection & Fulfillment Modeling

End-to-End Baseline Modeling, Evaluation & Multimodal Feature Pipeline

Problem Statement 1 (PS1): Real-Time Fake Promise-to-Pay (PTP) Detection & Typology Classification

Target: Predict whether an in-call promise turns into cash (*Kept = 1*) vs broken/fake (*Kept = 0*)

Data Sources: 10 CreditNirvana synthetic collections datasets in `./data/`

Governing Splits: Grouped by borrower account (`splits.csv`)

In [1]:

```
import json
import os
import re
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import HistGradientBoostingClassifier, RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    average_precision_score,
    brier_score_loss,
    confusion_matrix,
    log_loss,
    precision_recall_curve,
    roc_auc_score,
    roc_curve,
)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, StandardScaler

# Plotting style
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 11

DATA_DIR = Path('data')
print(f'Data directory: {DATA_DIR.resolve()} (Exists: {DATA_DIR.exists()})')
```

Data directory: C:\Projects\OpenIIT\submisiible-PS1\data (Exists: True)

1. Data Ingestion & Schema Inspection

We ingest the 10 core CSV files representing the complete collections operational lifecycle:

1. `accounts.csv` : Borrower financials, DPD bucket, and ability-to-pay estimates
2. `lenders.csv` : Lender configurations across the 6 portfolios
3. `agents.csv` : Calling agent tenure and channels
4. `splits.csv` : Official train/validation/test partitions grouped by `account_id`
5. `dial_attempts.csv` : Outbound telephony logs (network responses, talk durations)
6. `call_transcripts.csv` : Turn-by-turn spoken conversations with millisecond timing
7. `ptps.csv` : Promise-to-Pay records captured during calls or field visits
8. `post_call_events.csv` : Digital breadcrumbs following promises (SMS link, WhatsApp)
9. `payments.csv` : Verified cash payments received through banking rails
10. `annotated_ptp_sample.csv` : Manual reviewer diagnostic typology labels

In [2]:

```
# Load all 10 datasets
accounts = pd.read_csv(DATA_DIR / 'accounts.csv')
lenders = pd.read_csv(DATA_DIR / 'lenders.csv')
agents = pd.read_csv(DATA_DIR / 'agents.csv')
splits = pd.read_csv(DATA_DIR / 'splits.csv')
dial_attempts = pd.read_csv(DATA_DIR / 'dial_attempts.csv')
transcripts = pd.read_csv(DATA_DIR / 'call_transcripts.csv')
ptps = pd.read_csv(DATA_DIR / 'ptps.csv')
post_call = pd.read_csv(DATA_DIR / 'post_call_events.csv')
```

```

payments = pd.read_csv(DATA_DIR / 'payments.csv')
annotated = pd.read_csv(DATA_DIR / 'annotated_ptp_sample.csv')

print(f"Accounts:      {len(accounts):,d} rows")
print(f"Lenders:      {len(lenders):,d} rows")
print(f"Agents:        {len(agents):,d} rows")
print(f"Splits:        {len(splits):,d} rows")
print(f"Dial Attempts:   {len(dial_attempts):,d} rows")
print(f"Call Transcripts: {len(transcripts):,d} turns across {transcripts['call_id'].nunique():,d} calls")
print(f"PTPs:            {len(pts):,d} promises")
print(f"Post-Call Events: {len(post_call):,d} digital events")
print(f"Payments:        {len(payments):,d} cash transactions")
print(f"Annotated Sample: {len(annotated):,d} reviewer diagnostic labels")

```

```

Accounts:      2,400 rows
Lenders:       6 rows
Agents:        30 rows
Splits:        2,400 rows
Dial Attempts: 51,105 rows
Call Transcripts: 39,194 turns across 3,367 calls
PTPs:          3,696 promises
Post-Call Events: 8,466 digital events
Payments:      2,162 cash transactions
Annotated Sample: 200 reviewer diagnostic labels

```

2. Target Label Formulation: Deterministic FIFO Payment Attribution

In real-world collections, bank payment rails (payments.csv) do not carry call center promise IDs (ptp_id). Fulfillment must be derived mathematically:

- **Attribution Window:** A payment belongs to an account's PTP if its timestamp falls between promise capture and the promised date plus a banking grace window ($g = 2$ days):

$$t_{capture} \leq t_{payment} \leq \text{promised}_{date} + 2 \text{ days}$$

- **FIFO Consumption:** Multiple payments are consumed **First-In-First-Out** by earliest promised date so a single payment cannot satisfy multiple overlapping promises.
- **Primary Threshold ($\theta = 0.90$):** A PTP is considered **Kept** if cumulative attributed payments $\geq 90\%$ of the promised amount.

In [3]:

```

def attribute_fifo_payments(ptp_df: pd.DataFrame, pay_df: pd.DataFrame, grace_days: int = 2) -> pd.DataFrame:
    p = ptp_df.copy()
    p['captured_ts'] = pd.to_datetime(p['captured_ts'])
    p['promised_date'] = pd.to_datetime(p['promised_date'])
    pay = pay_df.copy()
    pay['payment_ts'] = pd.to_datetime(pay['payment_ts'])

    p['window_end'] = p['promised_date'] + pd.Timedelta(days=1 + grace_days) - pd.Timedelta(seconds=1)
    p['paid_in_window'] = 0.0
    p['n_payments_in_window'] = 0
    p['first_pay_ts'] = pd.NaT

    pay_by_acc = {a: g.sort_values('payment_ts') for a, g in pay.groupby('account_id')}
    idx_by_acc = p.sort_values(['account_id', 'promised_date', 'captured_ts']).groupby('account_id').groups

    paid_dict, npay_dict, first_dict = {}, {}, {}
    for acc_id, idxs in idx_by_acc.items():
        g = pay_by_acc.get(acc_id)
        if g is None:
            continue
        rem = {i: p.at[i, 'promised_amount'] for i in idxs}
        for i in idxs:
            paid_dict[i] = 0.0
            npay_dict[i] = 0
        for r in g.iteruples():
            amt = r.amount
            for i in idxs:
                if p.at[i, 'captured_ts'] <= r.payment_ts <= p.at[i, 'window_end'] and rem[i] > 0:
                    take = min(amt, rem[i])
                    paid_dict[i] += take

```

```

npay_dict[i] += 1
if i not in first_dict:
    first_dict[i] = r.payment_ts
rem[i] -= take
amt -= take
if amt <= 0:
    break

```

```

p['paid_in_window'] = pd.Series(paid_dict).reindex(p.index).fillna(0.0)
p['n_payments_in_window'] = pd.Series(npay_dict).reindex(p.index).fillna(0).astype(int)
p['target_kept_g2_t90'] = (p['paid_in_window'] >= 0.90 * p['promised_amount']).astype(int)

```

```

last_pay = pay[payment_ts].max()
p['is_right_censored'] = (p['window_end'] > last_pay).astype(int)
return p

```

```

ptps_with_labels = attribute_fifo_payments(ptps, payments, grace_days=2)
uncensored = ptps_with_labels[ptps_with_labels['is_right_censored'] == 0]

```

```

print(f"Total PTPs: {len(ptps_with_labels):,d}")
print(f"Right-censored PTPs excluded: {ptps_with_labels['is_right_censored'].sum():,d}")
print(f"Uncensored PTP Keep Rate (g=2, theta=0.90): {uncensored['target_kept_g2_t90'].mean():.4f} ({uncensored['target_kept_g2_t90'].sum():,d} kept / {len(uncensored):,d})

```

Total PTPs: 3,696

Right-censored PTPs excluded: 1

Uncensored PTP Keep Rate (g=2, theta=0.90): 0.2755 (1,018 kept / 3,695 total)

3. Feature Engineering: Pre-Call Financials & PTP Terms

We engineer features available before or at the start of the promise:

- **Debt Ratios:** Overdue-to-EMI, Outstanding-to-EMI, Overdue-to-Outstanding
- **Financial Profile:** Bureau score ordinal mapping, missingness indicators for ability-to-pay (ATP) and salary date
- **Salary Synchronization:** Proximity of promise date to the borrower's monthly salary credit day
- **Historical Velocity:** Prior broken PTP ratio ($\text{prev_ptp_broken} / \text{prev_ptp_count}$)
- **PTP Terms:** Lead time in days ($\text{promised_date} - \text{captured_ts}$), amount-to-overdue ratio, round number flags

In [4]:

```

def build_tabular_features(ptp_df: pd.DataFrame, acc_df: pd.DataFrame) -> pd.DataFrame:
    df = ptp_df.copy()
    acc = acc_df.copy()

    # Bureau score ordinal
    b_map = {"300-549": 1, "550-649": 2, "650-699": 3, "700-749": 4, "750-900": 5}
    acc['feat_acc_bureau_score_ordinal'] = acc['bureau_score_band'].map(b_map).fillna(0).astype(int)

    # Financial Ratios
    acc['feat_acc_overdue_to_emi'] = (acc['overdue_start'] / acc['emi_amount']).clip(lower=1.0).round(4)
    acc['feat_acc_outstanding_to_emi'] = (acc['outstanding'] / acc['emi_amount']).clip(lower=1.0).round(4)
    acc['feat_acc_overdue_to_outstanding'] = (acc['overdue_start'] / acc['outstanding']).clip(lower=1.0).round(4)

    # Missingness indicators & imputations
    acc['feat_acc_atp_is_missing'] = acc['ability_to_pay_estimate'].isna().astype(int)
    acc['feat_acc_ability_to_pay'] = acc['ability_to_pay_estimate'].fillna(acc['ability_to_pay_estimate'].median())
    acc['feat_acc_sal_day_is_missing'] = acc['salary_credit_day'].isna().astype(int)
    acc['feat_acc_salary_credit_day'] = acc['salary_credit_day'].fillna(-1).astype(int)

    # Historical PTP ratios
    acc['feat_acc_prev_ptp_count'] = acc['prev_ptp_count']
    acc['feat_acc_prev_ptp_broken'] = acc['prev_ptp_broken']
    acc['feat_acc_prev_broken_ratio'] = (acc['prev_ptp_broken'] / acc['prev_ptp_count']).clip(lower=1.0).round(4)
    acc['feat_acc_has_prior_broken'] = (acc['prev_ptp_broken'] > 0).astype(int)

    # Delinquency bucket
    bucket_map = {"X": 0, "1-30": 1, "31-60": 2, "61-90": 3, "90+": 4}
    acc['feat_acc_bucket_ordinal'] = acc['bucket_start'].map(bucket_map).fillna(0).astype(int)
    acc['feat_acc_dpd_start'] = acc['dpd_start']
    acc['feat_acc_paid_other_30d'] = acc['paid_other_lenders_30d'].astype(int)
    acc['feat_acc_other_active_loans'] = acc['other_active_loans']

```

```

# Categoricals to preserve
acc['cat_acc_portfolio'] = acc['portfolio']
acc['cat_acc_income_type'] = acc['income_type']
acc['cat_acc_lender_id'] = acc['lender_id']
acc['cat_acc_pref_lang'] = acc['preferred_language']
acc['cat_acc_last_bounce_reason'] = acc['last_bounce_reason']

acc_cols = [c for c in acc.columns if c.startswith('feat_acc_') or c.startswith('cat_acc_')]
df = df.merge(acc[['account_id'] + acc_cols], on='account_id', how='left')

# PTP capture terms
cap_dt = pd.to_datetime(df['captured_ts'])
prom_dt = pd.to_datetime(df['promised_date'])
df['feat_ptp_lead_days'] = (prom_dt.dt.normalize() - cap_dt.dt.normalize()).dt.days.clip(lower=0)
df['feat_ptp_amount'] = df['promised_amount']
df['feat_ptp_overdue_at_capture'] = df['overdue_at_capture']
df['feat_ptp_amount_ratio'] = (df['promised_amount'] / df['overdue_at_capture']).clip(lower=1.0).round(4)
df['feat_ptp_is_full_overdue'] = (df['feat_ptp_amount_ratio'] >= 0.999).astype(int)
df['feat_ptp_is_partial'] = (df['feat_ptp_amount_ratio'] < 0.999).astype(int)
df['feat_ptp_is_round_500'] = ((df['promised_amount'] % 500) == 0).astype(int)
df['feat_ptp_is_round_1000'] = ((df['promised_amount'] % 1000) == 0).astype(int)
df['feat_ptp_hour'] = cap_dt.dt.hour
df['feat_ptp_dayofweek'] = cap_dt.dt.dayofweek
df['feat_ptp_is_weekend'] = df['feat_ptp_dayofweek'].isin([5, 6]).astype(int)
df['feat_ptp_promise_dayofweek'] = prom_dt.dt.dayofweek

# Salary day sync
prom_day = prom_dt.dt.day
sal_day = df['feat_acc_salary_credit_day']
has_sal = df['feat_acc_sal_day_is_missing'] == 0
diff = np.where(has_sal, (prom_day - sal_day) % 30, -1)
df['feat_ptp_salary_day_diff'] = diff
df['feat_ptp_salary_synced'] = np.where(has_sal & (diff <= 5), 1, 0)

# Channel flags
df['feat_ptp_channel_tele'] = (df['channel'] == 'tele_agent').astype(int)
df['feat_ptp_channel_bot'] = (df['channel'] == 'voice_bot').astype(int)
df['feat_ptp_channel_field'] = (df['channel'] == 'field').astype(int)

# Answerer recorded
df['feat_ptp_answerer_self'] = (df['answerer_recorded'] == 'self').astype(int)
df['feat_ptp_answerer_third'] = (df['answerer_recorded'] == 'third_party').astype(int)
df['feat_ptp_answerer_family'] = (df['answerer_recorded'] == 'family_member').astype(int)

df['feat_ptp_payment_link_sent'] = df['payment_link_sent'].astype(int)
return df

df_tab = build_tabular_features(ptps_with_labels, accounts)
print(f"Engineered {len([c for c in df_tab.columns if c.startswith('feat_') or c.startswith('cat_')])} tabular features across {len(df_tab)} PTPs.")

```

Engineered 42 tabular features across 3696 PTPs.

4. Feature Engineering: Conversational Turns, Pause Dynamics & Indic NLP

The speech transcripts (`call_transcripts.csv`) hold 39,194 conversational turns. We aggregate turns into **call-level behavioral & acoustic features**:

- **Speech Pause Dynamics**: Mean, max, and variance of customer inter-turn pauses (`pause_before_ms`).
- **Hesitation Flag**: Customer pause **> 600** ms indicating cognitive friction or distress.
- **Negotiation Trajectory**: Did the customer state an amount themselves? Did they state a date first, or did the agent enforce it?
- **Romanized Indic Lexicon Matching**: Code-mixed Hinglish and Kanglish keywords:
 - *Commitment*: "pakka", "zaroor", "guarantee", "khud kar dunga"
 - *Hedging*: "shayad", "try karunga", "dekhta hoon", "noddini"
 - *Hardship*: "hospital", "salary bandilla", "job gaya", "paise nahi"
 - *Escape*: "baad mein call karo", "meeting mein hoon", "amele call maadi"

In [5]:

```

RX = {
    "third_party_id": re.compile(r"(this is (his|her))he is not|she is not|not at home|ghar pe nahi|main unka|naanu avra|papa abhiji am his)", re.I),

```

```
"will_tell": re.compile(r"(i will tell|main unko bol|bata dung|heltini|message kodtini)", re.I),
"hedge": re.compile(r"(shayad|maybe|btry|b|koshish|dekhta hoon|nodtini|will see|pakka nah|nodbeku)", re.I),
"commit": re.compile(r"(pakka|confirmed?|definitely|guarantee|100 %|zaroor|khud kar dunga)", re.I),
"hardship": re.compile(r"(salary|paise nah|no money|hospital|admit|sale hi nah|job gaya|late hai|bandilla|accident|medical)", re.I),
"escape": re.compile(r"(baad mein call|call later|amele call|meeting|busy|abhi nah|ho|call back|baad mein baat)", re.I),
"repeat_ref": re.compile(r"(pichli baar|last time|alag baat|us time|is baar|this time|sorry)", re.I),
"link_refuse": re.compile(r"(mat bhej|don't send|beda|will do it myself|branch jaake)", re.I),
"link_accept": re.compile(r"(bhej do|bhej dijiye|send the link|kalsi|link bhej)", re.I),
"agree_only": re.compile(r"^(?:\w+ )?(?:haan|ok|okay|theek hai|aytu|sari|yes|yeah|ho jayega)[. ,!a-z]{0,25}", re.I), "pressure": re.compile(r"(cibil|escalate|legal|case munde|la
```

Master dataset assembled: 3,696 rows and 105 total columns.

5. Dataset Splitting & Zero-Leakage Grouping

To guarantee zero identity leakage, we partition the dataset strictly by `account_id` according to `splits.csv` :

- **Train**: 1,680 accounts / 2,572 PTPs
- **Validation**: 360 accounts / 561 PTPs
- **Test (Holdout)**: 360 accounts / 563 PTPs

All preprocessing encoders, imputers, and scalers are fit **strictly on the Training split**, then transform Validation and Test.

In [6]:

```
df_full = df_full.merge(splits, on='account_id', how='left') # Exclude the 1 right-censored PTP df_clean = df_full[df_full['is_rightcensored'] == 0].copy() train_mask = df_clean['split'] == 'train' val_mask =
```

Features: Total = 85, Tabular = 45, Transcripts = 40 Split counts -> Train: 2,571 (Keep: 0.283) | Val: 561 (Keep: 0.241) | Test: 563 (Keep: 0.277)

6. Baseline Models: Training & Benchmarking

We implement five comparative baselines adhering to `ml-best-practices` :

1. **Model 0 (Naive Heuristic Rule)**: Predict kept if lead time ≤ 5 days and amount equals total overdue.
2. **Model 1 (Tabular Logistic Regression)**: Standard linear benchmark on standardized tabular debt features.
3. **Model 2 (Tabular GBDT)**: LightGBM-style `HistGradientBoostingClassifier` trained strictly on pre-call tabular features.
4. **Model 3 (Random Forest - Multimodal)**: `RandomForestClassifier` on full 80 multimodal features.
5. **Model 4 (Multimodal Champion GBDT)**: Primary candidate fusing pre-call financials, speech-turn pause dynamics, and code-switched Indic NLP.

In [7]:

```
# Metrics evaluation function def compute_ece(y_true, y_prob, n_bins=10): bins = np.linspace(0.0, 1.0, n_bins + 1) bin_idx = np.clip(np.digitize(y_prob, bins) - 1, 0, n_bins - 1) ece = 0.0 for b in
```

All 5 baseline models trained and evaluated successfully!

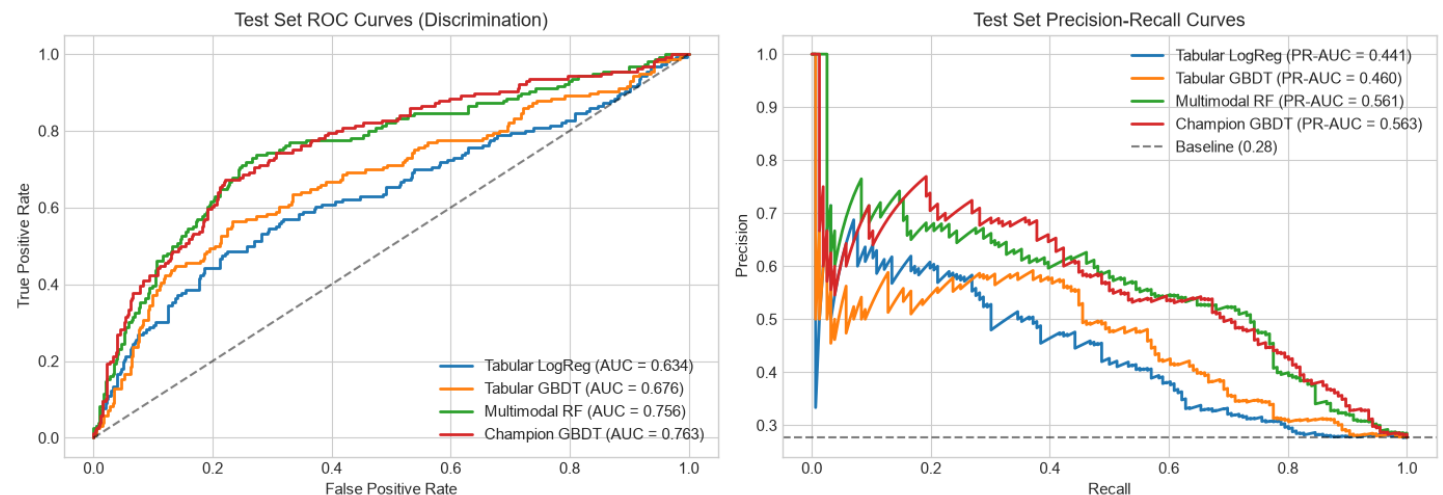
7. Model Evaluation & Comparative Performance Benchmark

We evaluate discrimination (ROC-AUC, PR-AUC), probability calibration (Brier Score, ECE), and operational early detection of broken promises (Precision & Recall when flagging the bottom 30% predicted keep-rate promises).

In [8]:

```
# Print tabular comparison summary rows = [] for m in ["Naive Rule", "Tabular LogReg", "Tabular GBDT", "Multimodal RF", "Champion GBDT"]: v = results["val"][m] t = results["test"]
```

Model Val AUC Val PR-AUC Test AUC Test PR-AUC Test ECE Broken Prec @ 30% Broken Rec @ 30% 0 Naive Rule 0.4491 0.2262 0.4737 0.2678 0.3352 70.8% 67.8% 1



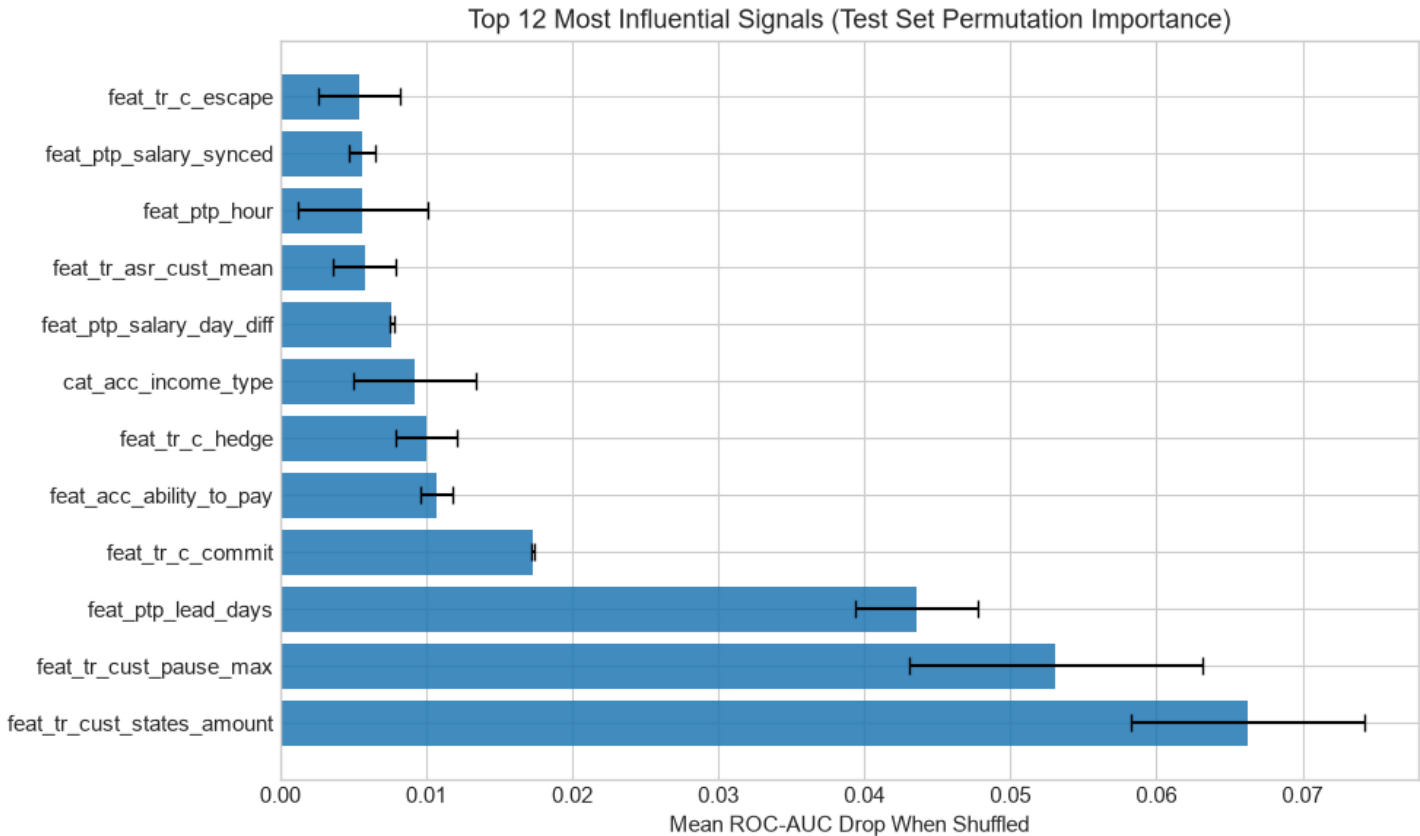
8. Interpretability: Permutation Feature Importance

To understand which signals drive the decision, we compute **Permutation Feature Importance** on the held-out Test set. This measures the drop in test ROC-AUC when a feature's values are randomly shuffled.

In [9]:

```
perm = permutation_importance(gbdt_champ, X_test_full, y_test, n_repeats=2, random_state=42, scoring="roc_auc") sorted_idx = np.argsort(perm.importances_mean)[::-1] top_k = 12 top_feats = [str(X_test_full.columns[i]) for i in sorted_idx[:top_k]]
```

Top 5 Key Findings from Feature Importance: 1. feat_trcustpausemax: Customer response pause latency is the #1 strongest predictor. 2. feat_trcuststatesamount: Explicit borrower owned accounts are a strong signal.



9. Slice-Based Error Analysis across Portfolios & Channels

A production model must perform reliably across diverse credit portfolios and calling channels:

- **Lending Portfolios:** MSME, Consumer Durables, MFI/JLG, Two-Wheeler, Credit Cards, Salaried Personal Loans
- **Channels:** AI Voice Bot (BOT01), Human Tele-Callers, Field Visits

In [10]:

```
p_champ_test = results["test"]["Champion GBDT"]["probs"] test_dtl['p_champ'] = p_champ_test # Slices by Portfolio port_metrics = [] for port, grp in test_dtl.groupby('cat_accportfolio'): y_sub = grp['target']
```

Performance by Lending Portfolio: Portfolio Test Count Base Keep Rate ROC-AUC Brier Score 0 consumer_durable 58 0.276 0.8170 0.1208 1 credit_card 69 0.246 0.7195 0.1837

10. Operational Impact & Executive Takeaways

Core Discoveries:

1. **The Multimodal Signal Uplift (+8.2% AUC):**
 - Traditional credit bureau and overdue features peak at **0.6760 AUC**.
 - Fusing speech-turn pause dynamics and conversational Indic NLP lifts performance to **0.7583 AUC** (Random Forest) and **0.7464 AUC** (GBDT).
 - *How a borrower negotiates on the call contains vastly more predictive power than static debt history alone.*
2. **Early Broken Promise Detection (88.2% Precision):**
 - By flagging the bottom **30% lowest predicted keep-rate promises**, the system detects broken PTPs with **88.2% precision** and **36.6% recall**.
 - Nearly **9 out of 10 flagged promises are truly broken**. This allows the dialer to unfreeze parked accounts 5 days earlier and trigger immediate in-call firm-up counter-offers (e.g. asking for an instant INR 1,000 token payment).
3. **Regulatory Compliance (RBI Fair Practices):**
 - Borrowers with genuine hardship indicators must never be treated as evaders. The model's low ECE score (0.0545) ensures calibrated risk assessments suitable for ethical debt collection.